Capability Embedding Project Report

CAPABILITY EMBEDDING FOR AI PLANNING

Project Report

Submitted By

Name: M.K NANDANA

Registration Number: TCR24CS047

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. System Architecture
6. Project Structure
7. Methodology
8. Implementation
9. Experiments
10. Results
11. Conclusion
12. Future Scope
13. References

---

1. Abstract

Artificial Intelligence planning systems require efficient methods for representing, comparing, and composing capabilities. This project presents a feature-based Capability Embedding framework in which each capability is represented as a numerical feature vector. Similarity measures such as Cosine Similarity and Euclidean Distance are used to evaluate relationships between capabilities. The framework supports capability compatibility analysis, capability composition, alternative implementation comparison, irrelevant capability detection, and operational attribute evaluation. Graphical visualizations including bar charts and heatmaps are generated to provide meaningful insights into the experimental results. The project demonstrates how numerical embeddings can improve the efficiency of capability analysis in AI planning systems.

---

2. Introduction

Artificial Intelligence planning focuses on selecting suitable actions to achieve predefined goals. Modern planning systems often contain a large number of capabilities, making manual comparison and analysis difficult.

Capability Embedding is an approach that converts capabilities into numerical vectors based on predefined functional and operational features. Once represented numerically, capabilities can be compared using mathematical similarity measures, allowing automatic compatibility analysis and capability composition.

This project implements a modular Capability Embedding framework using Python. The framework includes data representation, feature encoding, similarity computation, evaluation metrics, visualization, and experimental analysis.

---

3. Problem Statement

Traditional capability representations are mainly symbolic, making automatic comparison and composition difficult. As the number of capabilities increases, manual evaluation becomes inefficient.

The problem addressed in this project is the development of a feature-based embedding framework capable of:

- Representing capabilities numerically.
- Comparing capabilities using similarity metrics.
- Identifying compatible capabilities.
- Detecting unrelated capabilities.
- Evaluating operational characteristics.
- Visualizing analytical results.

---

4. Objectives

The main objectives of this project are:

- Design a feature-based capability representation.
- Convert capabilities into embedding vectors.
- Compute similarity using Cosine Similarity.
- Measure Euclidean Distance between capabilities.
- Compose compatible capabilities.
- Compare alternative capability implementations.
- Detect irrelevant capabilities.
- Evaluate operational properties such as execution time and reliability.
- Visualize experimental results using graphs and heatmaps.

---

5. System Architecture

The system consists of multiple independent modules working together to perform capability embedding and analysis.

The major components are:

- Data Module
- Models Module
- Embedding Module
- Similarity Module
- Experiment Module
- Evaluation Module
- Results Module

(Insert Architecture Diagram here.)

---

6. Project Structure

The project follows a modular directory structure.

```
CapabilityEmbedding/
│
├── data/
│   ├── capabilities.py
│   └── features.py
│
├── embeddings/
│   ├── encoder.py
│   ├── composer.py
│   └── similarity.py
│
├── evaluation/
│   ├── evaluator.py
│   ├── metrics.py
│   ├── graphs.py
│   └── visualizer.py
│
├── experiments/
│   ├── experiment1.py
│   ├── experiment2.py
│   ├── experiment3.py
│   ├── experiment4.py
│   └── experiment5.py
│
├── models/
│   ├── capability.py
│   ├── goal.py
│   └── state.py
```

│

├── images/

├── results/

├── report/

│

├── DESIGN.md

├── README.md

├── requirements.txt

└── main.py

(Insert Folder Structure Diagram here.)

---

7. Methodology

The methodology adopted in this project consists of the following steps:

1. Load capability definitions.
2. Load feature definitions.
3. Encode capabilities into feature vectors.
4. Compute similarity scores using Cosine Similarity.
5. Calculate Euclidean Distance.
6. Perform capability composition.
7. Execute experimental evaluations.
8. Generate graphs and heatmaps.
9. Analyze the results.

(Insert Workflow Diagram here.)

---

8. Implementation

The project is implemented using Python and follows a modular design.

Data Module

Stores capability definitions and feature information.

Models Module

Contains the Capability, Goal, and State classes.

Embedding Module

Responsible for encoding capabilities into numerical feature vectors.

Similarity Module

Calculates Cosine Similarity and Euclidean Distance between capability vectors.

Experiment Module

Performs five experiments to evaluate different aspects of capability embedding.

Evaluation Module

Generates graphs, heatmaps, and evaluation metrics.

---

9. Experiments

Experiment 1 – Capability Compatibility Analysis

This experiment evaluates whether two capabilities are compatible based on their feature vectors.

Result: Successfully identified compatible and incompatible capabilities.

Experiment 2 – Capability Composition

This experiment combines compatible capabilities into a single composite capability and compares it with the original capabilities.

Result: Successfully generated composite capability embeddings.

Experiment 3 – Alternative Capability Implementations

This experiment compares multiple implementations (API, GUI, and Database) of the same capability using similarity measures, execution time, and reliability.

Result: Successfully analyzed alternative implementations and generated comparison graphs.

Experiment 4 – Irrelevant Capability Detection

This experiment measures similarity between unrelated capabilities to identify irrelevant functionality.

Result: Successfully detected unrelated capabilities using similarity scores.

Experiment 5 – Operational Attribute Evaluation

This experiment compares capabilities using operational properties such as execution time and reliability.

Result: Successfully evaluated performance characteristics and generated visualization graphs.

---

10. Results

The project successfully generated:

- Capability Embedding Vectors
- Cosine Similarity Scores
- Euclidean Distance Values
- Composite Capability Embeddings
- Execution Time Comparison Graph
- Reliability Comparison Graph
- Similarity Comparison Graph
- Similarity Heatmap

The experimental results demonstrate that feature-based capability embedding provides an effective numerical representation for capability comparison and analysis.

---

11. Conclusion

This project successfully developed a feature-based Capability Embedding framework for AI planning. Capabilities were represented as numerical feature vectors, enabling compatibility analysis, similarity computation, capability composition, and operational evaluation. Experimental analysis and graphical visualizations demonstrated the effectiveness of the proposed approach. The modular implementation makes the framework easy to extend and maintain.

---

12. Future Scope

Future enhancements may include:

- Transformer-based semantic capability embeddings.
- Graph Neural Network (GNN) representations.
- Knowledge Graph integration.
- Reinforcement Learning for automated planning.
- Cloud-based capability repositories.
- Large-scale industrial planning applications.

---

13. References

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. Python Software Foundation. *Python Documentation*. https://docs.python.org/
3. NumPy Developers. *NumPy Documentation*. https://numpy.org/
4. Scikit-learn Developers. *Scikit-learn Documentation*. https://scikit-learn.org/
5. Matplotlib Developers. *Matplotlib Documentation*. https://matplotlib.org/